CS6868 Research Mini-Project

An Investigation into Parallel Task Scheduling using Dynamic Circular Work-stealing Deques (An OCaml Implementation)

Alina Banerjee
ns26z001@smail.iitm.ac.in

Ankita Das
cs25s032@cse.iitm.ac.in

Dhruvanshi Shah
ee22b143@smail.iitm.ac.in

May 1, 2026

Abstract

This project attempts to build a user-level task scheduler that can distribute tasks to dynamic circular deques. Using this scheduler, we run a set of parallelizable problems to measure the efficiency of work-stealing across such deques. Work-stealing is a paradigm where idle deques i.e. ones that have completed tasks allotted to them steal tasks from other deques, thus rebalancing loads across deques and speeding up overall program execution. At the same time, this introduces contention with possibly multiple deques trying to steal from one that may be about to pick up the same task for execution locally.

[1] originally introduced this idea of using array-based deques (double-ended queues) for managing tasks across multiprocessors. With fixed-size arrays, inefficient memory usage and array overflow were the fundamental problems. Our implementation is based on [2] which addressed both problems using dynamic cyclic arrays: arrays that can grow or shrink in size as needed and are cyclic which ensures the overflow problem is only constrained by the number of bits used for constructing these arrays.

We have implemented the deque and related data structures and built a parallel task scheduler to run a set of decomposable and hence parallelizable programs. We want to understand the behaviour of work stealing for different classes of problems and use them to measure the actual conditions under which work-stealing is most efficient.

1 Goals

The goals of the project are as follows:

1. To implement the Chase-Lev work-stealing deque in OCaml: [2] proposes a new data-structure: dynamic circular arrays that grow when the current array size is insufficient for adding newer items and shrink when the number of elements in the array is less than the array size by a specific factor. We want to port the basic methods presented in the paper – pop, push and steal – from Java to OCaml.

2. To implement a user-level parallel task scheduler: This scheduler will take a recursively computable problem and divide it into a set of initial tasks to be distributed across parallel scheduler work loops. The scheduler will support a fork command to break down the problem tasks into smaller-sized ones; to gather the computed results it will support a join command.

3. To develop accurate models for the deque to use for checking correctness in the linearizability of the implementation using QCheck-Lin and sequential specification using QCheck-STM.

4. To check that there are no data races in a the codebase, we will check all executions on an OCaml compiler instance augmented by TSAN (ThreadSanitizer).

   The following are the most interesting and possibly challenging parts of our goals:

   - ensure linearizability of the scheduler execution through ensuring correct behaviour in the growing, shrinking of the underlying circular arrays and ensuring correct deque behaviour - LIFO pushing and popping of tasks in a deque from its bottom end contends way with FIFO (stealing) of tasks from the deque from its top end.
   - building a parallel scheduler that operates correctly with deques that spawn, join and steal tasks concurrently and correctly. We have encountered both unicore (single-domain) and parallel round-robin schedulers (effect-based) with a single shared task queue as the work loop in CS6868 lectures 9 and 10 but without any work-stealing. Ensuring the correctness of work stealing by a single domain among several competing domains in the parallel scheduler with dynamic arrays will be the key.

   We evaluate three benchmark problems, categorised by their primary bottleneck:

   Compute-bound problems, where execution speed is primarily determined by the rate at which the CPU executes tasks: (1) Parallel Fibonacci, (2) Parallel map over an array.

   Memory-bound problems, where execution speed is primarily determined by the rate at which data is read from and written to memory: (1) Parallel merge sort,

   For each benchmark we measure the following metrics across both the work-stealing scheduler and a mutex-protected shared-queue scheduler: (1) parallel speedup relative to a sequential baseline, (2) steal frequency, (3) task throughput, (4) the impact of steal policy on steal frequency comparing round-robin and random victim selection, and (5) the crossover threshold below which sequential execution outperforms parallel execution.

   to understand the inflection points where work-stealing tasks are advantageous over other paradigms.

2 Background

A deque is a double-ended queue which can exhibit LIFO (stack) behaviour at one end and FIFO (queue) behaviour at another. Items can be pushed to and popped from one end (stack-like) whereas they can only be removed/popped from the other end in first-in-first-out order (queue-like).

Any parallelizable problem needs to be broken down into smaller units or tasks in order to execute tasks in parallel thus providing an advantage over sequential execution. In a multiprocessor system with several processes (domains for parallelization in OCaml), these tasks can be spread across them for faster execution. Once the set of initial tasks is spread across processes, the processes can themselves create new subtasks to be executed locally by the process.

[4] describes two paradigms on sharing tasks in a shared-memory system with multiple processors - work dealing and work stealing. Work-dealing involves work deques with more tasks than others giving away some of their tasks to other deques until the numbers of tasks are equalized between them. This requires both deques to stop what they are doing until task

sharing is complete. The determination on when to share is made when the number of tasks exceed a predetermined limit and requires multiple queues to stop and co-ordinate.

Work-stealing flips the onus on empty work deques to take tasks away from busier deques. With multiple possible empty deques, stealing brings contention to busier deques where multiple empty deques compete with each other to pick up tasks from a busy deque. A clever trick can alleviate this problem: of only stealing from one end of the deque (FIFO) to ensure that any synchronization need only be handled at that end (called the top).

The [1] implements a user-level task scheduler based on work-stealing array-based deques associated with processes as described above. The three main methods for the ABP deque are:

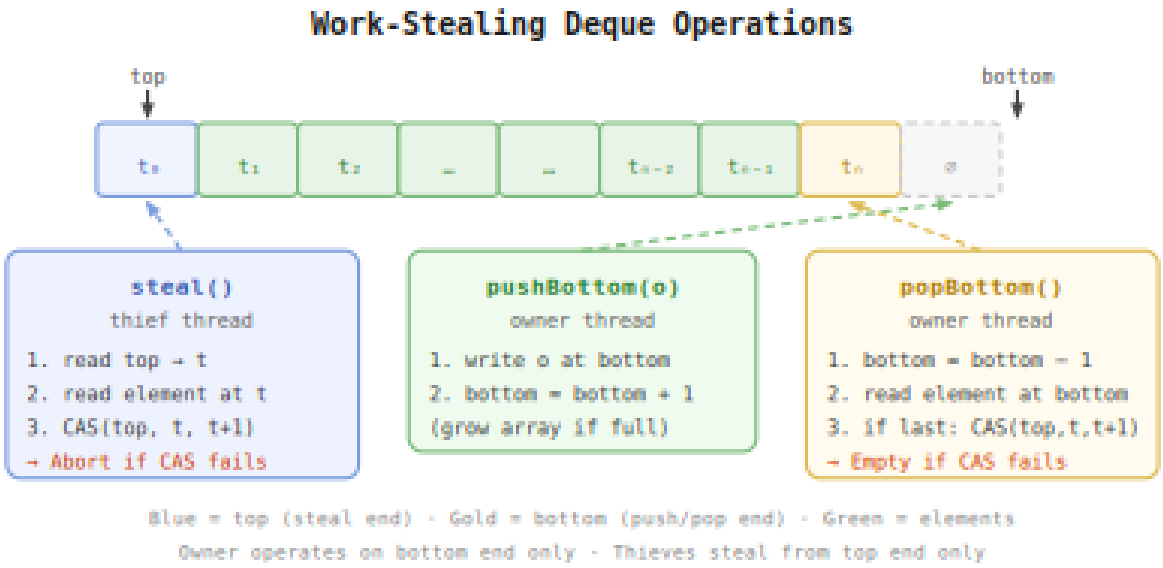

Figure 1: The three operations of the work-stealing deque.

- pushBottom o: which pushes o to the bottom end of the deque
- popBottom: pops an object from the bottom of the deque; if the deque is empty, an Empty value is returned.
- steal: if a stealing process attempts to deque a value from the local deque, one of the three values can be returned:
  - if a task is successfully stolen from the top end of the deque, then that task is returned
  - if the local deque is empty, then an Empty value is returned
  - if the stealing process loses out to another stealing process, then an Abort value is returned.

The salient feature of the ABP algorithm is that the arrays backing the deques are of fixed size. Changing these arrays to be cyclic - where the top and the bottom indices are reset to point to the beginning of the array when the deque becomes empty - makes overflow less frequent but does not rule out its occurrence. A subsequent design from [3] used a list of arrays to address this problem but these arrays were not cyclic and using the list of arrays presented a more complicated algorithm to reason about in terms of time and space complexity.

The Chase-Lev algorithm addresses the overflow problem by using dynamic cyclic arrays. The only practical restriction on deque size is integer overflow in the address space. In our

OCaml implementation, integers have a runtime representation of 63 bits, giving a maximum value of $2^{63} - 1 \approx 9.22 \times 10^{18}$.

Given the operation rate quoted in the paper of 4×10^9 operations per second, the time until deque index overflow occurs is:

$$\frac{2^{63} - 1}{4 \times 10^9} \approx 2.30 \times 10^9 \text{ seconds}$$

Converting to years, using $365.25 \times 24 \times 3600 \approx 3.156 \times 10^7$ seconds per year:

$$\frac{2.30 \times 10^9}{3.156 \times 10^7} \approx 73 \text{ years}$$

In practical terms, integer overflow is not a concern for this algorithm on modern systems.

In our implementation, deques are implemented using cyclic array whose ends are indicated by two indices where:

- top: is the first element at the top end of the deque. This end has first-in-first-out (FIFO) behaviour and is incremented on every steal operation.
- bottom: is the *next available* slot in the array where a new element will be pushed. This index is incremented on every `pushBottom` operation. This end of the deque had last-in-first-out (LIFO) behaviour and so operates like a stack.

The deque is empty when the `bottom` is less than or equal to `top`.

The deque size increases when items are pushed into it. If the current array is full, a new array with a specified larger size is created. The elements from the current array are copied over into this new, bigger array. Lastly, the new item is pushed into this expanded array. The inverse occurs when items are popped from the deque. If the number of elements in the deque is determined to be less than a heuristic based on the deque size relative to a predetermined constant, then the circular array is shrunk to a lesser size. Again, array elements are copied over to this new, smaller array with the popped element no longer in it.

Arrays backing deques are circular i.e. items are stored indexed modulo to the array size. On growing the array, moving items into the newer, larger-sized array, there is no need to update top or bottom although one must always keep in mind that the actual array indices in this array may be different.

The top and bottom indices operate on opposite ends of the deque and only contend when the last element can be popped by the local process or stolen by another process. For this, top is only ever modified by the `steal` operation and its value is never decremented indicating a linearized timeline of items stolen from one end of the deque. With the `top` index never decremented, no separate tag field is needed to be stored in the deque to solve the ABA problem that occured in earlier work-stealing algorithms.

3 Tasks Undertaken

3.1 Implementation

3.1.1 Deque and Circular Array implementation

The implementation began with translating the code for deque operations presented in the [2] paper from Java to OCaml (0b61520).

The deque stores all items in the active array data structure which is defined as an atomic reference as are the top and bottom indices. This is to ensure the absence of data races between local and stealing processes acting on the deque.

The `Atomic.compare_and_set` is used to ensure that only one among possibly multiple competing stealing processes can take an item off the top end of the deque. The process which succeeds returns with the item wrapped in a Value, others return an Abort indicating an aborted attempt to steal.

In `steal`, a thief increments `top` via a successful CAS, while the owner may concurrently decrement `bottom` in `pop_bottom`. The deque is empty when `top` $\geq$ `bottom`, and the only genuine race occurs when exactly one element remains — both operations attempt to claim it simultaneously, resolved by whichever CAS succeeds first.

Based on the code snippets in the [2] paper, the circular array only stores the log of the size, not the size itself. This makes the growing (260420a) and shrinking (2fe082a) of the circular array elegant and only requires copying items from the older to the expanded or the shrunk array.

The `Deque.perhaps_shrink` method shrinks the array if the number of elements in the deque is less than 1/K of the array size, where K >= 3. This ensures that when the array is shrunk and halved in size, the number of elements are less than 1/3rd of the array. This ensures that there's enough free space for newer items to be pushed to the deque and no need to expand the array soon after shrinking.

3.1.2 Scheduler implementation

The scheduler (609e0ef) whose constituent types are shown above is a parallel work-stealing one where each domain (worker) owns a local double-ended deque of tasks. Worker domains push and pop tasks from the bottom of their own deque in a lock-free manner, while idle workers choose another domain's deque to steal tasks from using a compare-and-swap (CAS) instruction.

This design ensures that synchronization cost is proportional to stealing frequency rather than task frequency, since the local push and pop requires no cross-domain coordination.

Each task receives a context (`ctx`) carrying a reference to the shared pool, which holds the array of per-worker deques, a count of pending tasks yet to complete, and a count of successful steals. Tasks are decomposed via `fork`, which wraps a computation in a new task, pushes it onto the owner's local deque, increments the pending count, and immediately returns a `future` — a placeholder that will be filled with the result when the task eventually executes. `join` blocks logically on the spawned task but keeps the domain productive by continuing to execute or steal tasks until the awaited future is filled. It runs the same work-stealing loop — popping from the local deque or stealing from a victim — until the awaited future is ready. This ensures that synchronisation cost is proportional to stealing frequency rather than task frequency: local `push_bottom` and `pop_bottom` require no cross-domain coordination, and the CAS is only invoked during a steal or when the last element of a deque is contested.

3.2 Testing and verification

3.2.1 QCheck-Lin testing

The QCheck-Lin test involved developing a model using the `Lin_domain.Make_internal` functor and types for deque commands and results.

The initial model was incorrect as all deque operations were spread across two parallel domains,

violating the deque's owner/thief constraint where only one domain can call `push_bottom` and `pop_bottom` while `steal` is called by the other domain in parallel. Also, returning a `Deque.Abort` value has no sequential equivalent causing spurious linearizability failures i.e. only if an actual Value is returned can a `pop_bottom` or an `steal` operation be successful. An empty deque or a failed steal attempt never actually return a value and must be treated as semantically equal results when concurrent operations on parallel domains are executed.

Running the test on a TSAN-enhanced opam switch did help trace and pinpoint an error (fixed in 0ecff4e). TSAN was throwing up a data race on the `push_bottom` operation. The problem was that `push_bottom` was reading the `active_array` i.e. the array backing the deque and potentially growing it. However, a concurrent steal operation reading the `active_array` between the `grow` operation and the `put_item` operation would have seen the older, stale array and read an element from it while the owner was writing to the new one. The fix was to ensure the array swap was published atomically via `Atomic.set` before any writes to the new array, so that any concurrent `steal` either sees the old array in its entirety or the new one — never an intermediate state.

3.2.2 QCheck-STM testing

The QCheck-STM test required a more careful design than the Lin test because STM checks exact values against a sequential model rather than just linearizability. The sequential model represents the deque as a plain int list where the front is the steal end (top index) and the back is the push/pop end (bottom index). `next_state` advances the model by appending to the back for the `push_bottom` operation, removing from the back for the `pop_bottom` operation, and removing from the front for the `steal` operation.

A major hurdle was navigating STM's GADT-based `res` type — the Res constructor wraps values alongside a `ty_show` combinator, and naive pattern matching inside the `postcond` caused OCaml's type checker to report ambiguous type escaping errors. The fix was to compute expected values as plain int option values outside the GADT match before comparing.

The second hurdle was the same owner/thief constraint from the Lin test - `STM_domain.Make` distributes all operations across both parallel domains, causing `push_bottom` and `pop_bottom` to race concurrently and crash with index out of bounds. The fix was to use the `arb_triple` with separate generators for each domain similar to the qcheck-lin test.

4 Evaluation

4.1 Experimental Setup

All benchmarks were run on a personal machine running macOS with an Apple M-series processor. The machine has four performance cores and four efficiency cores with a shared L2 cache. All measurements were taken using `OCaml 5.4` compiled to native code with optimisation flag `-O3`.

Each benchmark configuration was run 10 times after 3 warm-up runs whose results were discarded. To handle noise, we report the mean wall-clock time $\bar{t}$, the standard deviation σ, and the minimum observed time $t_{\min}$ across the 10 measured runs. Wall-clock time is measured using `Unix.gettimeofday` which has microsecond resolution on the test machine. Each run uses a freshly generated random array as input to avoid cache effects from reusing the same data across runs.

All benchmarks are run for both the work-stealing scheduler and the mutex-protected shared-queue scheduler described in Section 3, under varying worker counts (1, 2, 4, 8, 16) and task

granularity thresholds

4.2 Fibonacci benchmarks

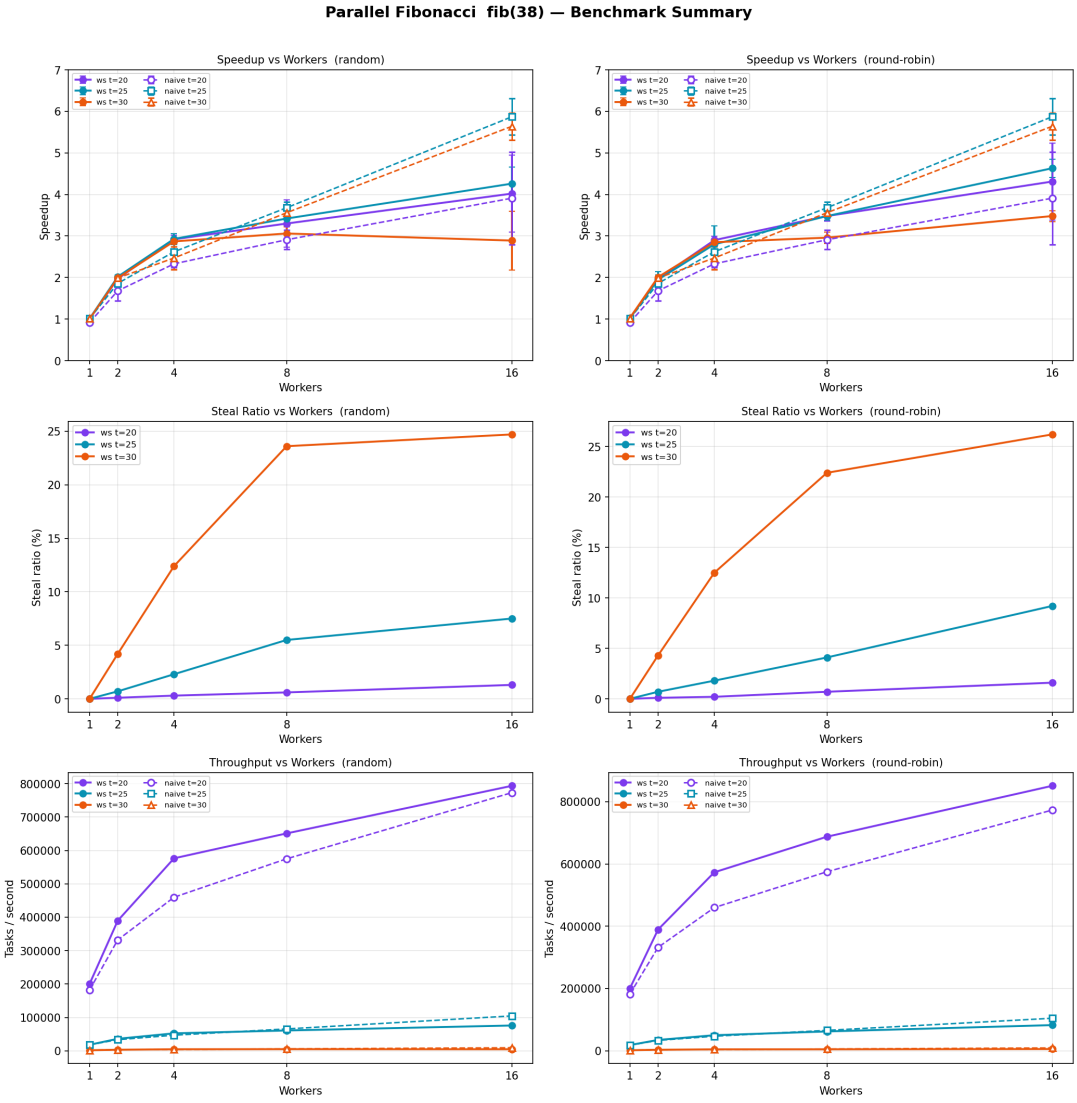

Figure 2: Fibonacci benchmarks
Figure 2

The takeaways from our benchmarking are:

- The parallel Fibonacci benchmark reveals that task granularity has a more significant effect on the naive scheduler than on work-stealing. At coarse granularity (threshold=30), work-stealing achieves modest speedup of around 3× at 16 workers while naive reaches nearly 6 times - the small task count means workers rarely contend on the mutex and the queue distributes work efficiently.

- At fine granularity (threshold=20), the situation reverses: work-stealing reaches 4 times speedup while naive drops to under 4 because the high task count causes heavy mutex contention on every push and pop. The steal ratio plots confirm this – at threshold=30 up to 26% of tasks are stolen, reflecting genuine load imbalance as workers exhaust their

local queues quickly, while at threshold=20 stealing stays below 2% since each worker has abundant local work.

- The throughput plots expose the same effect from a different angle: threshold=20 generates more tasks/second for work-stealing but naive cannot keep up at that granularity, whereas threshold=25 and threshold=30 show naive matching or exceeding work-stealing throughput because the larger task sizes amortize the mutex overhead.

The results confirm that work-stealing is more robust across thresholds — its speedup varies from 3 to 4 times regardless of granularity — while naive is highly sensitive to task size, performing well only when tasks are large enough to make the mutex cost negligible.

4.3 Mergesort benchmarks

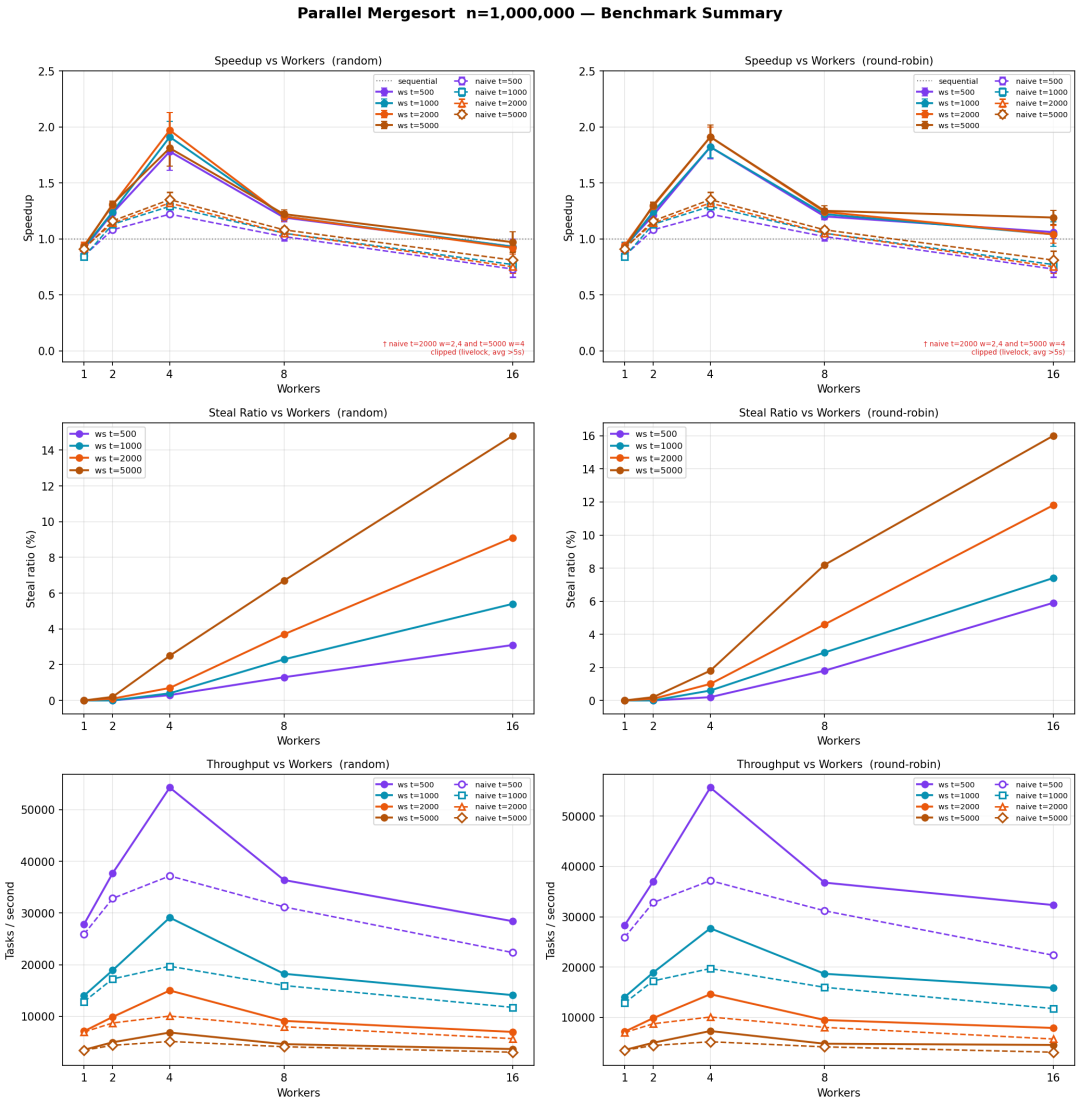

Figure 3: Mergesort benchmarks
Figure 3

The takeaways from our benchmarking are:

- All configurations peaking at 4 workers and degrading beyond that point. Work-stealing

achieves a peak speedup of around 2 at 4 workers — a meaningful improvement over the naive scheduler which peaks at around 1 at the same worker count – demonstrating that dynamic rebalancing is beneficial for the unbalanced merge tree where top-level tasks take orders of magnitude longer than leaf tasks.

- Beyond 4 workers both schedulers degrade, with work-stealing (random) falling below sequential at 16 workers while round-robin holds marginally above 1, and naive dropping to 0.73–0.81 at 16 workers as mutex contention compounds memory bus pressure. The steal ratio rises consistently with both threshold and worker count, reaching 16% at threshold=5000 with 16 workers under round-robin – higher than random at the same configuration (14.8) – reflecting the structural imbalance of the merge tree driving idle workers to steal large pending operations.
- The throughput plots show work-stealing at threshold=500 achieving 55,000 tasks per second at 4 workers before degrading, while naive throughput at the same granularity is consistently lower and follows a similar peak-and-drop pattern, both confirming that 4 physical performance cores is the effective parallelism limit for this memory-bound workload.

4.4 Parallel map benchmarks

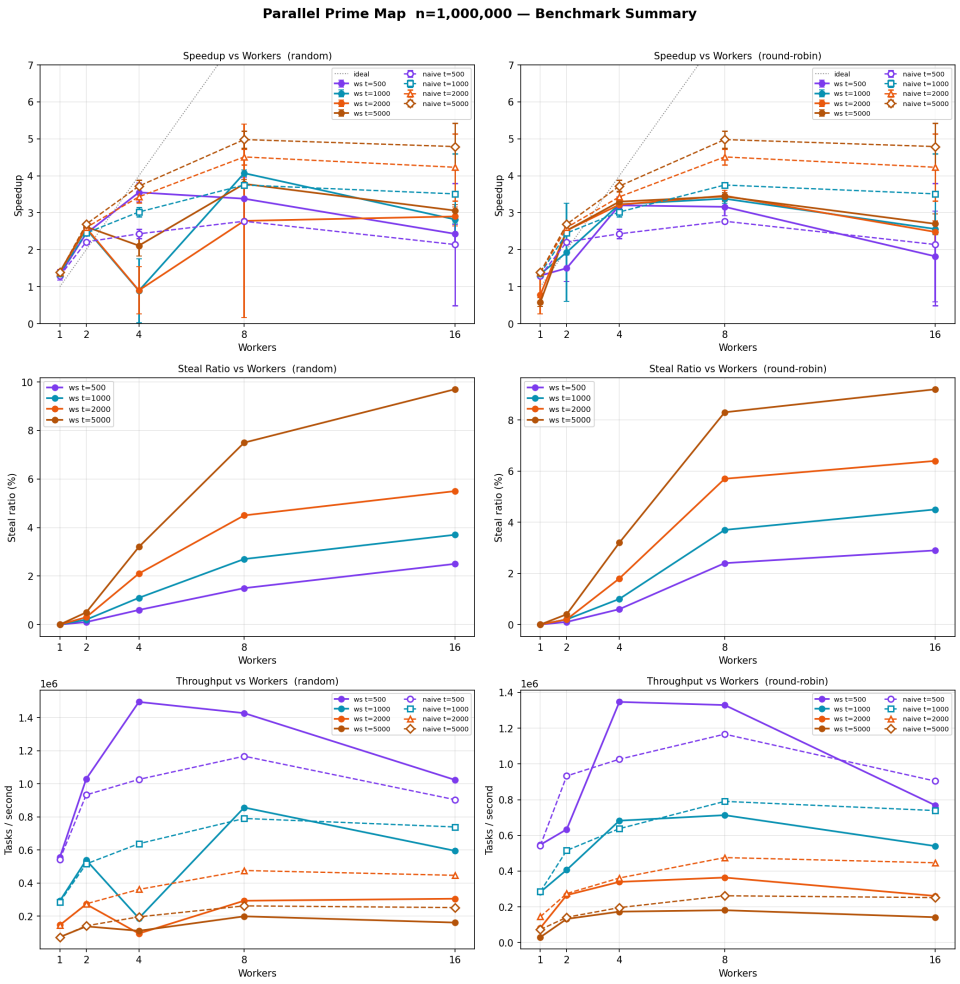

Figure 4: Parallel map benchmarks
Figure 4

The takeaways from our benchmarking are:

- The naive scheduler with coarse granularity (threshold=5000) reaches nearly 5 times the speedup at 8 workers and continues improving to 4.8 at 16 workers – consistently beating every work-stealing configuration beyond 4 workers. This occurs because the naive queue distributes work evenly without any rebalancing, and larger tasks fully amortize the mutex overhead.
- Work-stealing shows high variance at 4 and 8 workers in the random policy panel – visible as large error bars – reflecting the same livelock instability seen in mergesort when many fine-grained tasks compete for stealing. The steal ratio plots confirm the benchmark is genuinely balanced: even at `threshold=5000` with 16 workers the steal ratio stays just below 10%, well below the mergesort steal ratio of 14 to 16% at the same worker count, confirming that uniform chunk costs keep workers productive locally without frequent stealing.
- The throughput plots are the most striking – naive at threshold=500 achieves over 1 million tasks per second at 4 workers and sustains this through to 16 workers, while work-stealing at the same threshold peaks at 1.5 million tasks per second at 8 workers then degrades, suggesting that beyond the optimal worker count the overhead of failed steal attempts on an already well-balanced workload costs more than it saves. THe round-robin scheduler shows notably lower variance than random in the throughput plots, trading peak throughput for stability – a pattern consistent across all three benchmarks.

5 Reflection on the Use of LLMs

Tools and models used. Claude's Sonnet 4.6 (Adaptive) model was the only one used for the following: (1) translating the deque and circular array code from Java to OCaml (2) working out the design of scheduler especially contexts and pools and subsequent implementation; also debugging errors when benchmark programs were run. (3) working out and implementing the qcheck-lin and qcheck-stm tests, especially informing us of the *arb_triple* function to test parallel deques (3) writing the code for the benchmark programs (4) writing the Python scripts for generating all the output plots (5) cleaning up some LaTeX code

What worked well. In terms of helping work out functions, module designs etc., Claude acted as a teacher in prompting us to think a bit first on what was needed, try to write the necessary code and help come up with a correction or completely new piece if one was stuck. In terms of the qcheck test, it was crucial in helping figure out the design for testing deques in parallel with limitations on their actions (i.e. a deque can never steal from itself). Claude did away the major tedium and time it would have taken to painstakingly write benchmark programs and plot the outputs.

What did not work. Minor things such as suggesting OCaml's `Atomic.fetch_and_add` when `Atomic.incr` or `Atomic.decr` is perfectly fine. The non-adaptive model for Sonnet made a mistake when asked to generate an example from the paper - it thought `pop_bottom` to occur from the top index of the deque, an impossibility.

What was surprising. The patience with which it would help think through something. This is our first major project where an LLM has been used to think through and work out a research

paper, so that was helpful and pleasant. Claude was surprisingly very good at helping work out the scheduler design and implementation but needing help simplifying basic functions was unexpected.

What was difficult. While Claude helped in the translation of paper's code for deques from Java to OCaml, the order for writing to/reading from arrays, correctly updating their indices had to be carefully reasoned about. While testing one or two errors cropped up in these functions and having worked through the code step-by-step helped debug it.

Your overall assessment. While Claude was crucial in helping work through major parts of and design the project in a short time, it would be in our long-term benefit to approach the problem more slowly, look at existing codebases, struggle a bit more, only then approach a tool such as Claude because it can be very dangerous to use it as an easy resource to come up with answers without really grappling with the problem in the first place.

6 Conclusions

The three benchmarks validate the core claims of [2] while revealing important nuances about when work-stealing's advantage materialises in practice. We confirm that the Chase and Lev algorithm is correct under the assumption that the owner is the only domain operating on the bottom end of the deque — our QCheck-Lin and QCheck-STM tests formally verify this with owner/thief invariants. The steal ratio data directly confirms the paper's central efficiency argument: across all benchmarks and configurations at least 74% of tasks execute locally without cross-domain synchronisation, with stealing rates as low as 0.1% for fine-grained balanced workloads. For memory-bound unbalanced workloads such as mergesort, work-stealing achieves nearly 2 times speedup at 4 workers compared to the naive scheduler's 1.35 times – dynamic rebalancing allows idle workers to claim large top-level merge tasks from busy ones, precisely the irregular workload scenario the paper's dynamic array design was intended to address. Beyond 4 workers both schedulers degrade as memory bandwidth becomes the bottleneck, with the naive scheduler falling to 0.73 times at 16 workers as mutex contention compounds memory bus pressure.

For statistically balanced workloads such as prime map and coarse-grained Fibonacci, the naive scheduler reaches 5 times speedup at high worker counts, confirming that work-stealing's rebalancing mechanism adds overhead without benefit when tasks distribute themselves evenly. Task granularity is the most important practical tuning parameter: work-stealing is robust across thresholds because the steal mechanism compensates for imbalance at any granularity, while the naive scheduler is highly sensitive — too fine a granularity saturates the mutex and too coarse risks livelock, exactly the brittleness that motivated the Chase-Lev design.

The paper's decision to store `log_size` rather than `size` directly, and to use monotonically increasing `top` and `bottom` indices that wrap cyclically through the array, proved elegant in the OCaml port — growing and shrinking reduce to single-bit increments and decrements of the exponent, and the monotonic `top` index eliminates the ABA problem that required a tag field in the earlier ABP algorithm of [1]. The paper notes in Section 4 that correct memory reclamation of shrunk arrays requires a shared buffer pool to prevent a thief from reading a freed array — OCaml's garbage collector makes this concern irrelevant, since the GC ensures no array is reclaimed while any domain still holds a reference to it, significantly simplifying the implementation. ThreadSanitizer caught a subtle array publication ordering race in `push_bottom` that matches precisely the class of memory ordering hazard the paper's Java

implementation relies on `volatile` to prevent — in OCaml 5 the fix was to ensure `Atomic.set` on `active_array` happens before any `put_item` call on the new array, providing the same publication safety guarantee. Random victim selection proved more stable than round-robin across all benchmarks, consistent with the paper's choice of randomised stealing as the default policy, particularly at high worker counts where round-robin's deterministic cycling creates contention hotspots. Taken together the results confirm that the Chase-Lev algorithm's design philosophy — optimise the common case of local push and pop, make stealing correct with a single CAS, and let the GC handle reclamation — translates cleanly and efficiently to OCaml 5.4's domain-based parallelism model.

With more time, profiling tools for cache and memory analysis would have helped characterise the memory shrinking and expansion behaviour in existing benchmarks; additional benchmarking programs would have broadened the evaluation. We attempted an implementation of circular array allocation from a shared buffer pool as described in Section 4 of [2], combined with the shrink policies from Section 3. Some initial outputs are available (6df58e8). A few observations and their possible explanations follow:

1. Using the pool allocator reduces total memory allocation (`minor_words`) compared to GC mode across all shrink policies, indicating the pool reuses backing memory rather than allocating fresh arrays.
2. The `no_shrink` policy grows only during warmup until peak deque capacity is reached, after which allocation pressure drops, so GC pressure is front-loaded. The peak array gets promoted to the major heap due to its long lifetime, which explains the significantly elevated major word count for `no_shrink`.
3. The `multi` policy aggressively shrinks to the lowest suitable size, causing grow–shrink oscillation and a high total number of grows. Despite potentially lower instantaneous memory usage, cumulative allocation (`minor_words`) is therefore high: the motivation of releasing unused capacity is undermined by the pool needing to grow back almost immediately.

This work also raises questions about how the number of grow – shrink cycles, combined with the choice of shrink policy, impacts GC allocation pressure and total memory allocated. When comparing algorithms, it is therefore important to consider not only throughput and instantaneous memory usage but also how the GC manages the resulting memory patterns.

We stress that this implementation requires refinement and that we would like to continue working on this in the near future. Comparing such an implementation against an equivalent set of schedulers (work-stealing, naive) in Rust using `crossbeam` or our own deque implementation — would help us understand acquire-release semantics much better as well as the practical differences between garbage-collected and manually-managed memory systems.

Contributions

Member	Contribution (%)	Main responsibilities
Alina Banerjee	34 %	deque, circular array, scheduler implementation, qcheck tests
Ankita Das	33 %	shared pool memory buffer implementation with different versions of shrinking deque arrays
Dhruvanshi	33 %	benchmarking, plotting and problem results generation
Total	**100%**	

References

[1] Nimar S. Arora, Robert D. Blumofe, and C. Greg Plaxton. Thread scheduling for multiprogrammed multiprocessors. In *Proceedings of the 10th Annual ACM Symposium on Parallel Algorithms and Architectures*, SPAA '98, pages 119–129. ACM, 1998. doi:10.1145/277651.277678.

[2] David Chase and Yossi Lev. Dynamic circular work-stealing deque. In *Proceedings of the 17th Annual ACM Symposium on Parallelism in Algorithms and Architectures*, SPAA '05, pages 21–28. ACM, 2005. URL: https://dl.acm.org/doi/10.1145/1073970.1073974, doi:10.1145/1073970.1073974.

[3] Danny Hendler, Yossi Lev, Nir Shavit, and Robert Tarjan. A dynamic-sized nonblocking work stealing deque. *Distributed Computing*, 18(3):189–207, 2005. doi:10.1007/s00446-005-0144-5.

[4] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2nd edition, 2020. URL: https://www.elsevier.com/books/the-art-of-multiprocessor-programming/herlihy/978-0-12-415950-1.